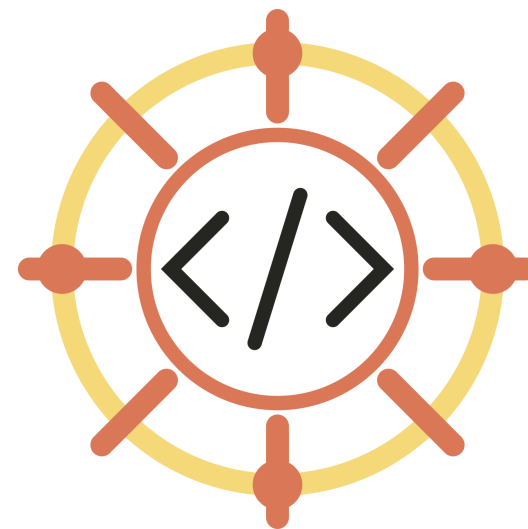


Anthropic

两万亿 IPO 的想象

编码模型与 AGI



01

Claude 为什么强

02

编码模型如何成功

03

为什么 AGI 需要编码

如果估值迈向两万亿，市场在押注什么？

不只是更好用的开发工具，而是更大范围的数字工作能力。

\$2T?

估值前景的情景讨论

从工具收入，到工作能力的价值

开发工具

写代码、修 Bug、跑测试

数字工作

对账、文件处理、跨系统操作

智能研发

运行实验、构建评测、优化基础设施

程序员是第一批用户；模型的行动能力，正在进入更多工作。

Opus 5.5 的成绩单：工程与通用行动

终端执行、综合推理、代码变更、多文件任务：四个能力切面。

59.6%

**Terminal-Bench
4.0**

AA · 终端执行 · max /
fallback

58

Intelligence Index

AA · 综合指数 v4.3.2

54.4%

FrontierCode 1.1

官方 · 代码变更是否可合并

57.8%

CursorBench 4.0

官方 · 真实会话多文件任务

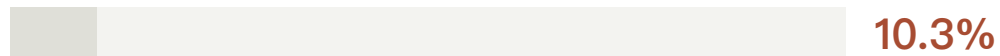
它不只补全代码：搜索、修改、执行和验证，已经进入同一个任务。

10.3% → 70.6%：一次明确的终端能力跃迁

Sonnet 5 → Sonnet 5.5 · Terminal-Bench 4.0 · 厂商公布结果。

完成终端任务 · 官方评测

Sonnet 5



Sonnet 5.5



+60.3

个百分点

读、改、跑、检查
在任务中连续发生

模型的进步，不只是答案更像人，而是更多系统任务能够被解决。

第三方对照：终端执行接近，综合能力有差异

Opus 5.5 (max / fallback) · GPT-6 Astra (max) · Gemini 3.1 Pro (Preview)

Terminal-Bench 4.0 · 平均 pass@1

统一 mini-swe-agent · 66 题 · 每题 3 次

Opus 5.5  59.6%

GPT-6 Astra  59.1%

Gemini 3.1 Pro  4.0%

AA Intelligence Index · 综合能力

v4.3.2 · 10 项评测 · 指数分，非百分比

Opus 5.5  58

GPT-6 Astra  53

Gemini 3.1 Pro  30

终端观测分差：仅 0.5 个百分点

指数反映综合能力，不是编码成功率

统一测试配置，才能比较分数；综合能力不等于终端任务成功率。

Sonnet 与 Opus：日常迭代和复杂工作的两种选择

Claude 5.5 · API 价格为美元 / 百万 Token；速度为 AA 测量输出速率。

模型	输入	输出	缓存读取	输出速率
Sonnet 5.5	\$2	\$10	\$0.20	132.3 t/s
Opus 5.5	\$4	\$20	\$0.20	92.0 t/s

Sonnet：清晰、频繁的日常任务

Opus：开放、复杂、持续判断

模型分工可以降低调用门槛；实际成本仍取决于一项任务用了多少 Token。

每次调用便宜，不等于整个任务更便宜

Artificial Analysis Coding Agent Index v1.5 · 平均每任务 API 费用。

Sonnet 5.5



14.20 美元

Opus 5.5



13.00 美元

GPT-6 Astra



7.47 美元

单任务费用

输入 + 缓存 + 输出
× 多轮执行

Claude Code / Codex · max

比较“效果、耗时与费用”的组合，比只看 Token 单价更接近真实使用。

重复上下文读取，价格降低 90% / 95%

缓存读取分别比普通输入便宜 90% / 95%；输出与写入另外计费。

模型	输入 \$ / M Token	缓存 \$ / M Token	费用降低
Sonnet 5.5	\$2.00	\$0.20	90%
Opus 5.5	\$4.00	\$0.20	95%

算例：20 轮 × 10 万 Token 相同前缀

Sonnet \$4 → \$0.40 Opus \$8 → \$0.40

缓存把反复阅读代码背景的成本降下来，让多轮工作更容易负担。

终端原生：同一个任务里，查、改、测、再修复

例：支付请求偶发失败，定位重试逻辑并准备补丁。

一次目标 → 多步动作

搜索	<code>rg "retry timeout" src/</code>
读取	查看支付调用与异常分支
修改	调整可重试错误与次数上限
验证	<code>pytest tests/test_payment_retry.py</code>
恢复	失败用例 → 修正 → 再运行

可检查的交付

补丁 + 回归测试

修改范围、测试结果
仍待确认的问题

人负责业务取舍与合并

CLI 的价值是动作可组合、反馈可读取，不是让 AI 拥有无限权限。

为什么不是“换个模型”就能复制体验？

工程能力来自模型判断与执行系统的共同作用。

环节	具体工程资产	解决的问题
模型判断	相关调用链、错误假设	下一步查哪里、改什么
工具接口	文件读取、精确补丁、终端回显	让动作可执行、失败可定位
上下文	CLAUDE.md、相关函数、最近报错	保留任务需要的信息
持续状态	功能清单、进度文件、Git、测试	中断后可恢复、下一会话可接班

从“回答正确”到“交付可靠”，还隔着接口、状态与验证。

长上下文，不等于找对这次任务的证据

修复支付重试：当前上下文需要的是关联信息，而不是全部文件。

全量灌入

支付 + 库存 + UI + 营销 + 配置



按问题探索

测试入口 → 复现条件

支付调用 → retry 分支

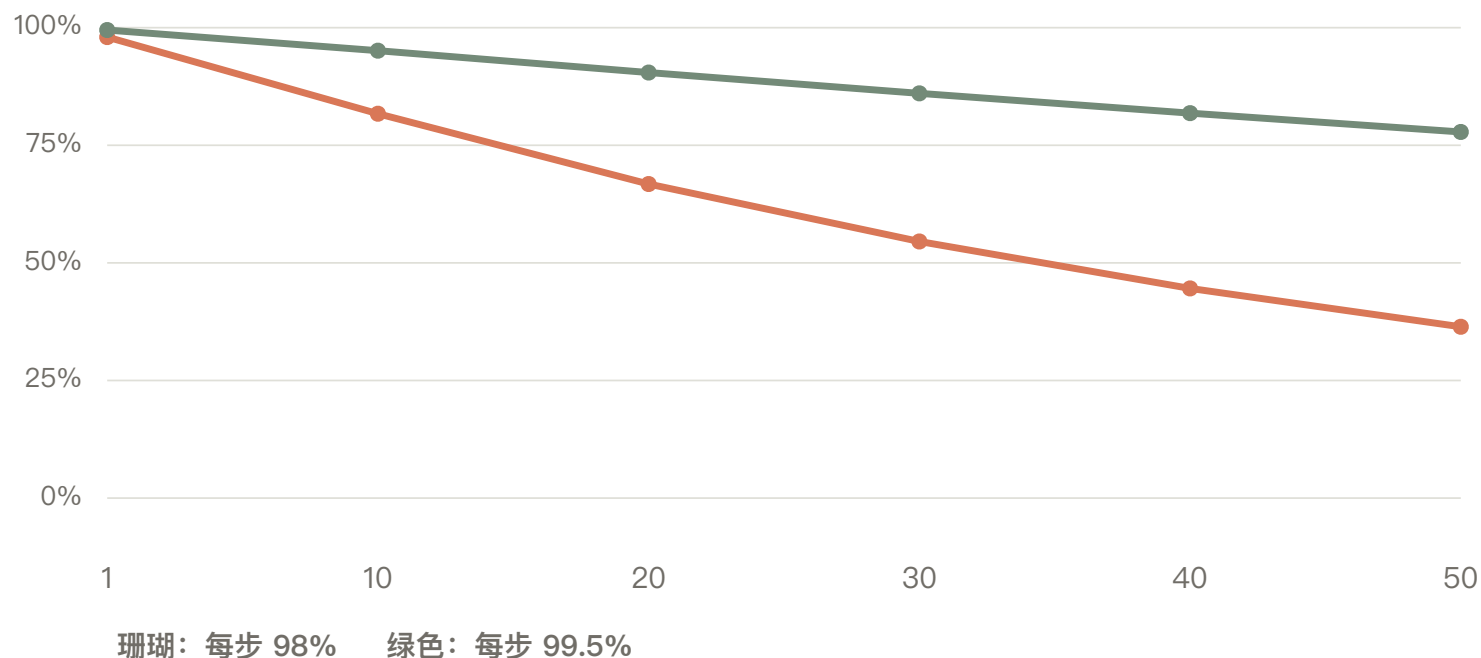
异常类型 → 超时配置

失败用例 → 修改后的验证

长窗口提供空间；主动检索决定哪些信息值得进入工作记忆。

每步只差 1.5 个百分点，50 步之后差多少？

数学算例：每步独立、可靠性恒定、失败不可恢复。



36.4%

→ 77.8%

50 步全部成功概率

真实体验的关键：少犯错、出错能恢复、少让人接管。

代码训练场有任务、有环境，也有自动裁判

可运行的环境 + 可检查的结果，让“做对了”不再只靠主观评价。

500

Verified

人工校验的 Issue 任务

642

Pro V2

离线执行的复杂仓库任务

66

Terminal-Bench 4.0

校准资源与超时的终端任务

代码库 + 容器环境 + 测试 / 验证器 → 可重复的执行反馈

编码的优势：大量试错能够运行、能够评分，也能够进入下一轮改进。

模型学习的，不只是一份正确补丁，而是整段解题轨迹

公开研究展示：强化学习能鼓励自我检查、反思与更有效的推理。



执行时修好这一题；训练时让下一批题的解题策略变得更好。

一道真实任务：不停机，从 MySQL 迁到 PostgreSQL

Terminal-Bench 4.0 · live-database-cutover · 迁移期间读写继续。

验收维度	测试动作	必须满足
不中断服务	切换期间持续重放客户请求	零失败、零丢行、零陈旧读取
行为保持	比较 API 状态码与响应内容	与 MySQL 基线一致
性能预算	检查各接口 p95 延迟	比基线最多增加 10 ms
迁移完成	验证数据、外键与后续写入	新部署也不能依赖 MySQL

裁判检查的是系统行为达标，不是配置文件或代码看起来正确。

Codex 的竞争力，不只在分数，也在执行效率

Artificial Analysis · Codex 两种模型配置 · 当前任务均值。

Codex 模型配置	编码 Agent 指数	平均运行时间	平均 API 费用
GPT-6.1 Sol · xhigh	63	15.5 分钟	\$1.04
GPT-6 Astra · max	62	29.4 分钟	\$7.47

指数 v1.5: DeepSWE 1.1 + Terminal-Bench 4.0 + SWE-Atlas-QnA

赛道已经从“谁答得更好”，进入“谁能以更低代价持续完成任务”。

普通员工的任务，也包含数据结构、循环与规则

对账示例：订单明细与银行流水；保留差异证据，不直接修改账目。



输出：异常清单 + 原始行号 + 核对依据

Agent 编写清洗与匹配脚本，人核对业务规则与异常解释。

MCP + 代码执行：先发现工具，再按需组合

把工具定义按需加载，让大规模工具接入不挤满上下文。

150,000

全量加载工具定义

Token · 官方工程示例

2,000

按需发现与读取

Token · 同一工程示例

—98.7%

这部分上下文用量

探索 `./servers/` → 读取目标工具接口 → 写脚本 → 在环境里执行

CLI 执行动作，MCP 接入系统，Skills 保存可复用的做事方法。

同一个模型，在不同工作上仍有不同表现

Opus 5.5 · 工具、科学、业务流程与桌面操作的评测切面。



¹ Chartography · with tools ² OSWorld 2.1 · partial

能力不会
均匀展开

检验具体工作
比套用一个总分
更能判断可用性

编码打开了通用行动的空间；业务适配与可靠性仍然决定落地效果。

26% 的内部研发工作，达到 AI 主导级别

Anthropic R&D Automation Index · 2026 年 8 月快照 · AL4 仍由人监督。

26%

AI 主导

高层目标输入 · 人类监督

>90%

至少 AI 协作

AL3 及以上 · 含 AL4

任务级别

AL3 · 协作

AL4 · 主导

AL5 · 完全自主

AL4 是协作及以上的一部分；完全自主仍未在所测任务类别中报告。

AI 已经从研究成果，变成研究能力的一部分。

约 3 万个 Agent，在同一平台并行做研发

Anthropic 最常用内部平台 · 2026 年 8 月快照。

≈ 30,000

同时运行的 Agent

单一内部平台 · 研究与工程

100%

执行前经过在线监控

平台内动作覆盖率

> 10 亿

被分析的决策

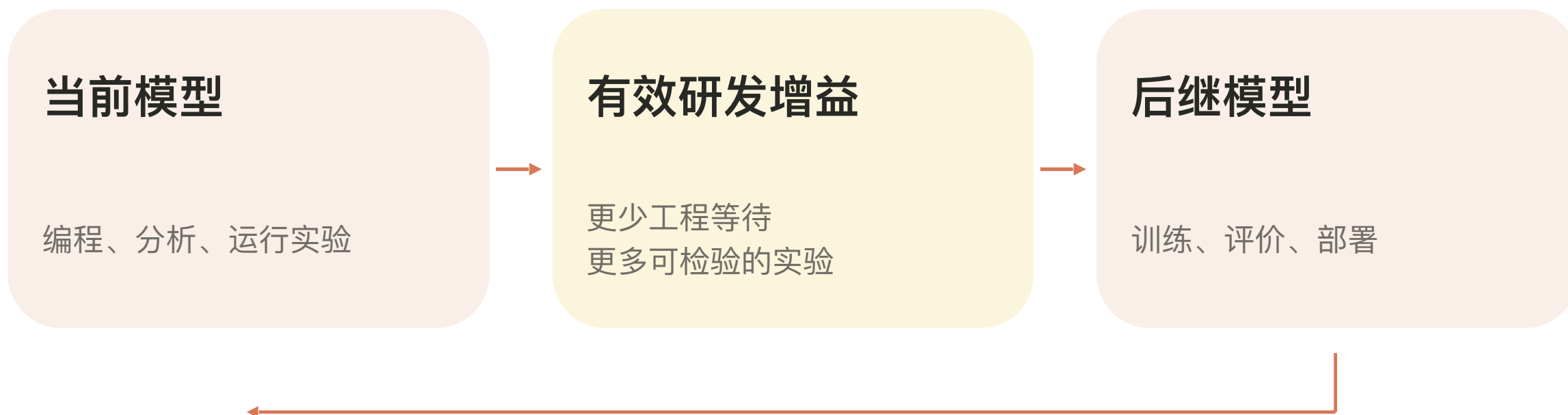
2026 年 8 月

大规模并行执行，把代码、实验与检查工具变成持续运转的研发系统。

这不是完全自主研究，而是人类设定目标、Agent 执行、系统持续监督。

当前智能参与研发，后继智能再进入下一轮

RSI：改进工具的能力，也开始被工具自身放大。



最震撼的不是机器替我们工作，而是参与制造下一代更会工作的机器。

把业务经验变成可执行能力，而不只是个人熟练度

先定义目标与规则，再委托执行；用产物和证据判断是否完成。

岗位	可以交给 Agent 的产物	人应握住的判断
运营 / 市场	指标计算、交叉分析、异常报告	问题是否值得问、指标是否正确
财务 / HR	对账清单、材料结构化、来源索引	规则、例外、金额与敏感信息
产品 / 研发	可运行原型、补丁、回归测试	需求价值、架构与上线责任

对公司：重新审视排期、交接和等待，释放过去做不起的任务。